

CASO PRÁCTICO 1 UD 2

José Cruces Aparicio - 2º DAM Interfaces

CONTROL DE DISPOSITIVOS DOMÉSTICOS CON INTERFACES NATURALES

1. Implementa la clase mando de televisión que debes usar desde nuestras interfaces naturales. Se aporta en los anexos.
2. Implementa una Clase llamada MandoVoz que permita subir y bajar el volumen del mando a través de la Voz cuando digas “subir volumen” y cuando digas “bajar volumen”. Para ello, utilizarás Vosk.

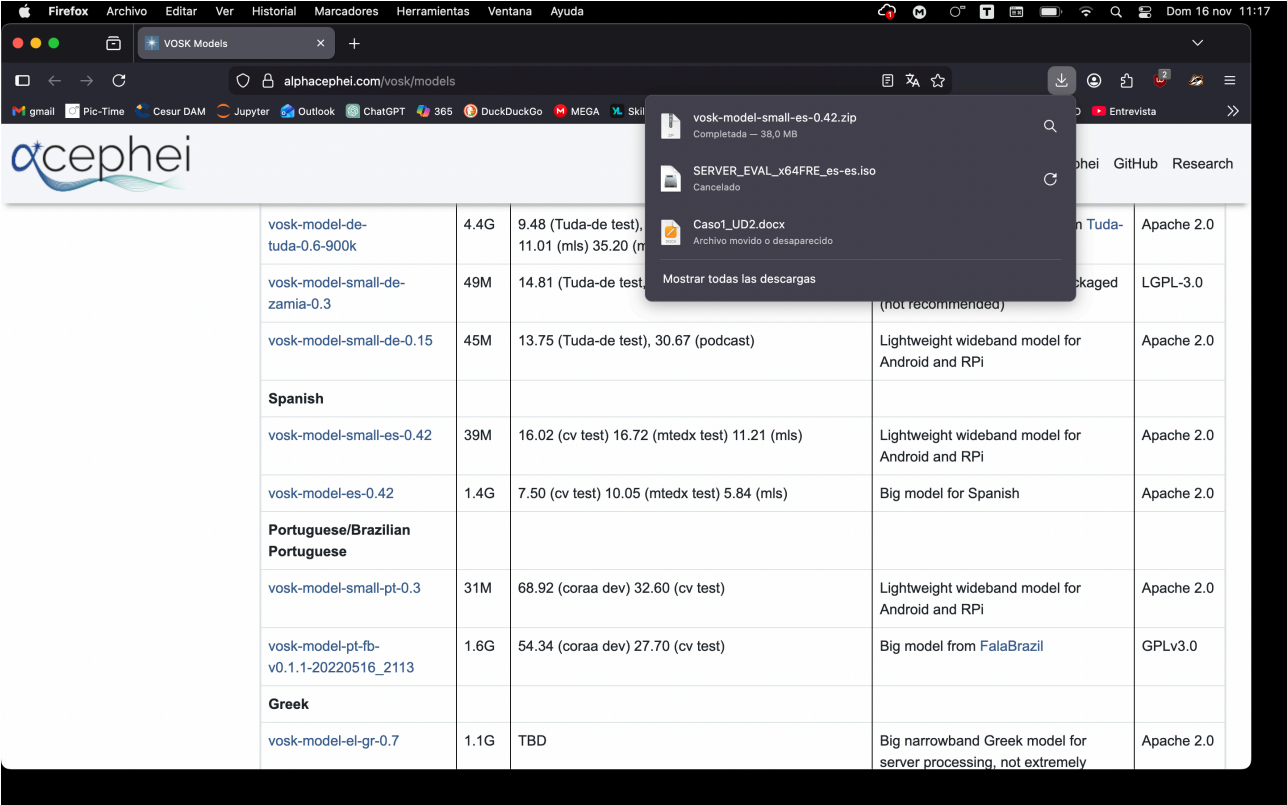
```
Comenzando a procesar audio...  
Comando recibido: Subir volumen  
Volumen actual: 1  
Comando recibido: Subir volumen  
Volumen actual: 2  
Comando recibido: Bajar volumen  
Volumen actual: 1  
Comando recibido: Subir volumen  
Volumen actual: 2  
Comando recibido: Subir volumen  
Volumen actual: 3  
Comando recibido: Subir volumen  
Volumen actual: 4
```

3. Implementa una Clase llamada DeteccionMovimientoAR que te permita subir el volumen cuando subas el brazo izquierdo y bajar el volumen cuando subas el brazo derecho. Funcionará como un entorno de realidad aumentada ya que podrás ver tu entorno y el volumen del mando. Para ello, utiliza javacv y opencv, tal y como se aporta en los anexos el código para detectar cuando sube el brazo izquierdo o derecho.



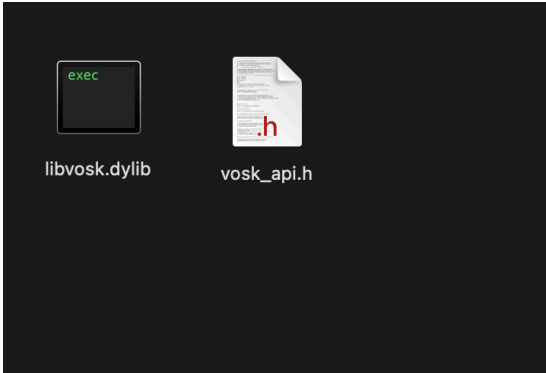
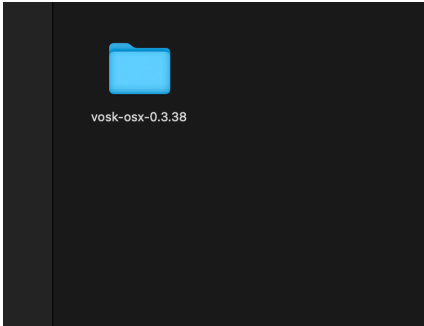
Se recomienda utilizar Java Maven para el desarrollo de este proyecto.

Para realizar este proyecto como recomienda el ejercicio lo primero que hemos hecho es crear un proyecto en IntelliJ con el nombre MandoAR con Maven, hemos descargado Vosk en su página web el modelo de reconocimiento de voz de español el ligero de tamaño, y además hemos descargado tras muchas pruebas en cuanto al SDK que no me diera problemas el Zulu-21 para x86 pese a ser un ARM puesto que el de ARM no funcionaba pero este sí perfectamente, ha sido un proceso el de los SDK y las dependencias bastante largo y tedioso imagino que por ser en Mac pero el resultado es de pleno funcionamiento. Aquí muestro capturas de pantalla de lo descargado, la creación del pom.xml y la configuración del Project Structure con el SDK escogido.

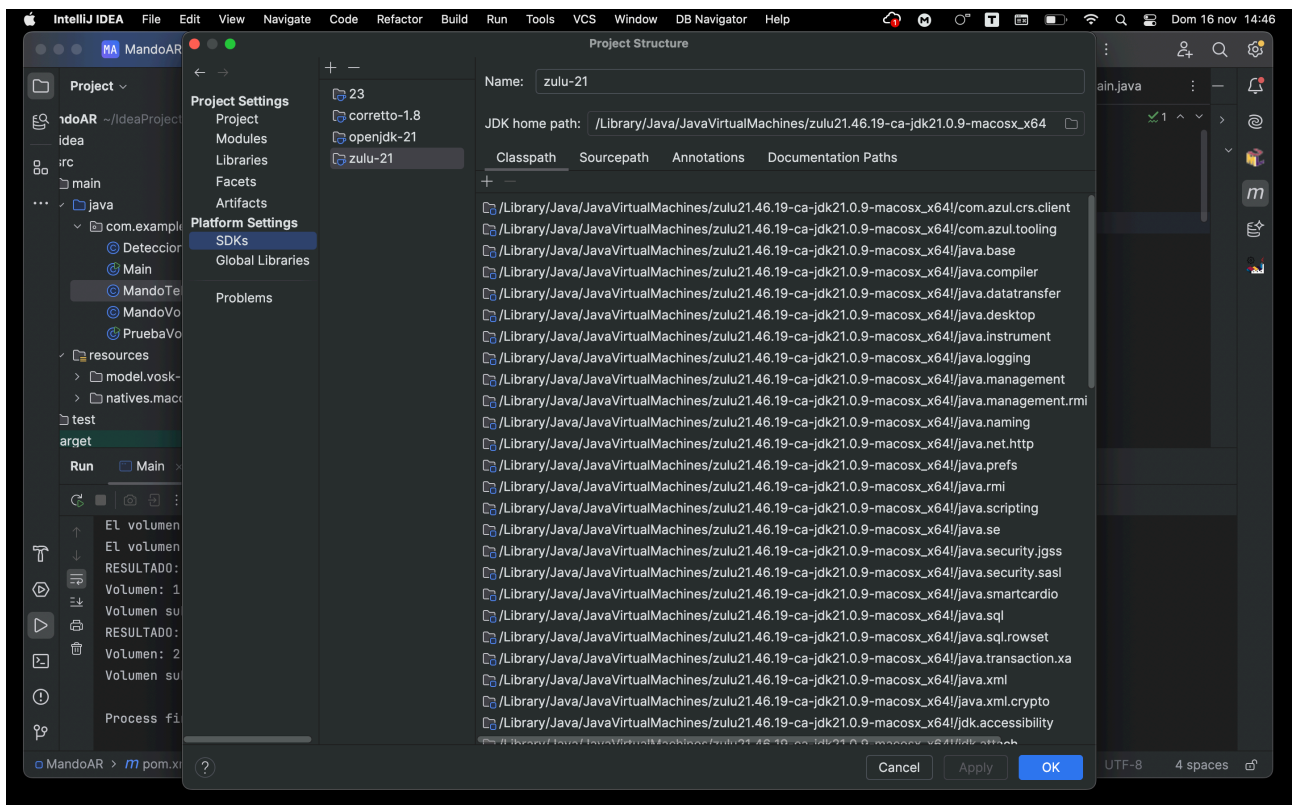
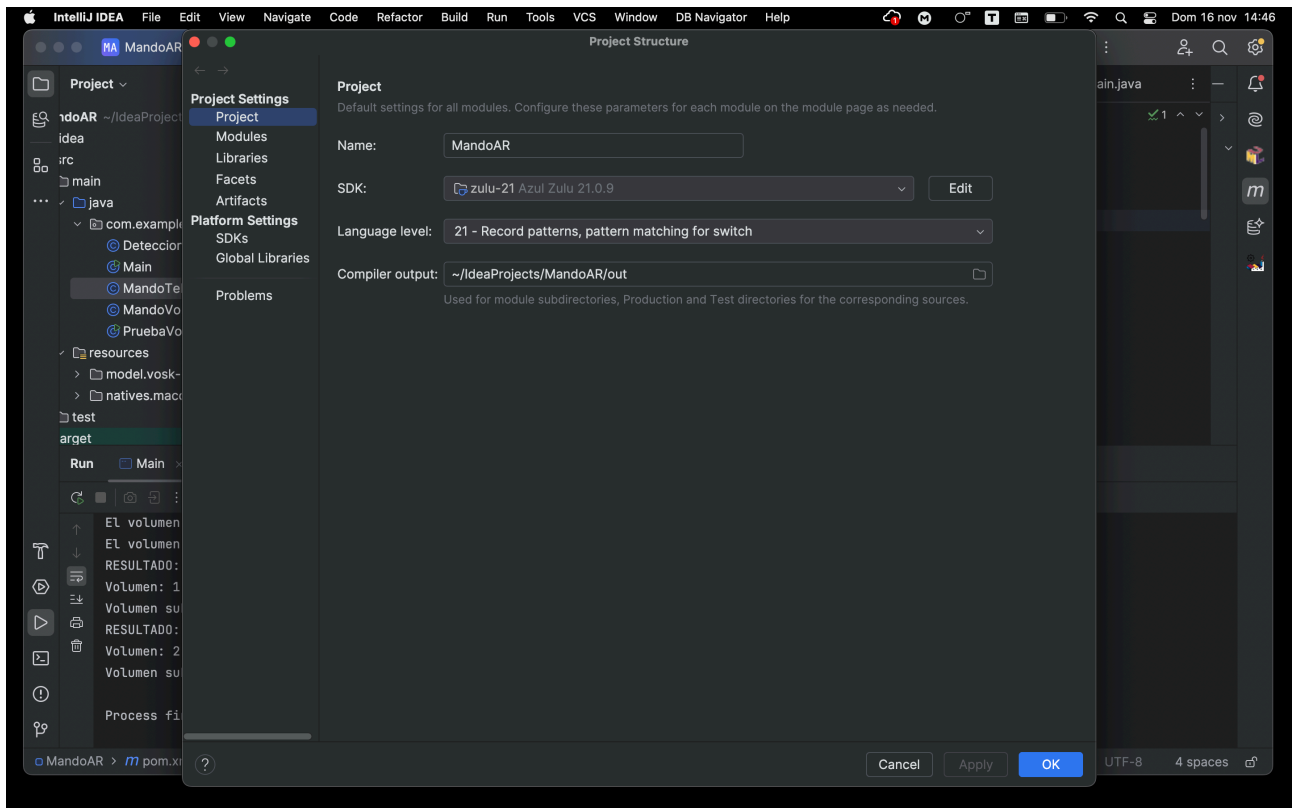


	vosk-model-de-tuda-0.6-900k	4.4G	9.48 (Tuda-de test), 11.01 (mls) 35.20 (m)			
	vosk-model-small-de-zamia-0.3	49M	14.81 (Tuda-de test)			
	vosk-model-small-de-0.15	45M	13.75 (Tuda-de test), 30.67 (podcast)		(not recommended)	
	Spanish					
	vosk-model-small-es-0.42	39M	16.02 (cv test) 16.72 (mtedx test) 11.21 (mls)		Lightweight wideband model for Android and RPi	Apache 2.0
	vosk-model-es-0.42	1.4G	7.50 (cv test) 10.05 (mtedx test) 5.84 (mls)		Big model for Spanish	Apache 2.0
	Portuguese/Brazilian Portuguese					
	vosk-model-small-pt-0.3	31M	68.92 (coraa dev) 32.60 (cv test)		Lightweight wideband model for Android and RPi	Apache 2.0
	vosk-model-pt-fb-v0.1.1-20220516_2113	1.6G	54.34 (coraa dev) 27.70 (cv test)		Big model from FalaBrazil	GPLv3.0
	Greek					
	vosk-model-el-gr-0.7	1.1G	TBD		Big narrowband Greek model for server processing, not extremely	Apache 2.0

Aquí muestro los iconos de lo descargado y como traslado la carpeta del modelo a su ubicación para la utilización que es la carpeta: `MandoAR/src/main/resources/natives/macosX/universal/`. En esa carpeta hay dejar los archivos directamente sueltos por que si dejas la carpeta no los reconoce bien.



Aqui muestro la captura de pantalla de como configuro el Project Structure con la SDK correcta



Lo primero que hacemos es dejar bien puestas las dependencias de nuestro proyecto editando el archivo /MandoAR/Pom.xml .

Aqui muestro el codigo de Pom.xml con las dependencias necesarias para que funcione todo lo que vamos a necesitar Vosk, JAN Javacv, OpenCV, con el modelo que descargamos :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>MandoAR</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- Vosk para reconocimiento de voz -->
    <dependency>
      <groupId>com.alphacephei</groupId>
      <artifactId>vosk</artifactId>
      <version>0.3.38</version> <!-- He a 0.3.38 porque la otra version no funciona -->
    </dependency>

    <!-- Añado JNA por que Vosk lo necesita para cargar su motor nativo -->
    <dependency>
      <groupId>net.java.dev.jna</groupId>
      <artifactId>jna</artifactId>
      <version>5.14.0</version>
    </dependency>

    <!-- JavaCV y OpenCV para detección de gestos -->
    <dependency>
      <groupId>org.bytedeco</groupId>
      <artifactId>javacv-platform</artifactId>
      <version>1.5.10</version>
    </dependency>
    <dependency>
      <groupId>org.bytedeco</groupId>
      <artifactId>opencv-platform</artifactId>
      <version>4.9.0-1.5.10</version>
    </dependency>

  </dependencies>
</project>
```


El siguiente paso es la creación de las clases que vamos a utilizar en el proyecto :

Main.java

MandoTelevision.java

DeteccionMovimientoAR.java

MandoVoz.java

Main.java

```
// Paquete para agrupar todas las clases que vamos a utilizar en nuestro proyecto
package com.example.mandoar;

// Importamos CountdownLatch, que utilizaremos para sincronizar y esperar a que los hilos de voz
y gestos terminen
import java.util.concurrent.CountDownLatch;

public class Main {
    public static void main(String[] args) {
        // Creamos el objeto mando para luego usarlo en el control de voz y en los gestos
        MandoTelevision mando = new MandoTelevision();

        // Latch para coordinar el cierre
        CountDownLatch latch = new CountDownLatch(1);

        // HILO 1: Control de voz
        Thread hiloVoz = new Thread() -> {
            MandoVoz controlVoz = new MandoVoz(mando);
            // gestionamos con el try & catch la ejecución y gestión de los errores
            try {
                controlVoz.iniciarReconocimiento();
                // gestión de las excepciones
            } catch (Exception e) {
                System.err.println("Error reconociendo la voz:");
                e.printStackTrace();
            } finally {
                latch.countDown(); // Notifica que el hilo de control de voz ha finalizado su ejecución
            }
        }, "Hilo-Voz");

        // HILO 2: Control por gestos
        Thread hiloGestos = new Thread() -> {
            DeteccionMovimientoAR controlGestos = new DeteccionMovimientoAR(mando);
            // gestionamos con el try & catch la ejecución del método de gestión de gestos y los errores
            try {
                controlGestos.iniciarDeteccion();
            } catch (Exception e) {
                System.err.println("Error en detección de gestos:");
                e.printStackTrace();
            } finally {
                latch.countDown();
            }
        }, "Hilo-Gestos");

        // Iniciar la detección de voz y gestos
        System.out.println("Iniciando MandoAR: Voz + Gestos");
        // Arrancamos los dos hilos
        hiloVoz.start();
```

```

hiloGestos.start();

// Esperamos con latch.await() a que uno de los hilos ( voz o gestos) termine su ejecución
try {
    latch.await();
    // Y aqui gestionamos las excepciones
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}

// Si el otro hilo sigue activo, se envia una señal de interrupción para detenerlo
if (hiloVoz.isAlive()) {
    hiloVoz.interrupt();
    System.out.println("Hilo de voz interrumpido.");
}
if (hiloGestos.isAlive()) {
    hiloGestos.interrupt();
    System.out.println("Hilo de gestos interrumpido.");
}
// Indicamos que el mando de realidad aumentada acaba de parar y mostramos el volumen final
// que hemos dejado
System.out.println("MandoAR finalizado. Volumen final: " + mando.obtenerVolumen());
}
}

```

Como comento a lo largo de todo el codigo el main es la clase principal y la encargada estructurar todo el funcionamiento del programa , llamando a los hilos que vamos a utilizar y también gestionando su salida y las excepciones . Parte del codigo utilizado en 3 de las 3 clases fue ya dado por el ejercicio pero he añadido al codigo los cambios necesarios para cumplir con los requisitos , los comento en el codigo y tras cada clase aclaro mas detalles.

MandoTelevisión.java

```

package com.example.mandoar;

public class MandoTelevision {
    // Creamos la variable volumen donde se alojara el valor del volumen
    private int volumen;

    // Creamos la variable volatile para que sea leida de la memoria principal no un cache del
    // procesador para que asi si se cambia su valor, ambos tanto hiloVoz como hiloGestos lo sepa

    private volatile boolean salir;

    public MandoTelevision() {
        this.volumen = 0;
        this.salir = false;
    }

    public void subirVolumen() {
        this.volumen++;
        System.out.println("Volumen: " + this.volumen);
    }

    public void bajarVolumen() {
        if (this.volumen > 0) {

```

```

        this.volumen--;
        System.out.println("Volumen: " + this.volumen);
    } else {
        System.out.println("El volumen ya está en 0 y no se puede bajar más.");
    }
}

public int obtenerVolumen() {
    return this.volumen;
}

//Añado el set y el get de salir para utilizarlo para cerrar la ejecucion del programa
public void setSalir(boolean salir) {
    this.salir = salir;
}

public boolean getSalir() {
    return this.salir;
}
}

```

Es el encargado de tener los getter y setter de variables y metodos a utilizar por la clase **DeteccionMoviminetoAR.java** y por **MandoVoz.java** .

DeteccionMoviminetoAR.java

```

package com.example.mandoar;

import org.bytedeco.opencv.opencv_core.Point;

import org.bytedeco.javacv.*;
import org.bytedeco.opencv.opencv_core.*;
import org.bytedeco.opencv.global.opencv_imgproc;
import org.bytedeco.opencv.global.opencv_core;
import org.bytedeco.opencv.opencv_videoio.VideoCapture;

import javax.swing.*;

public class DeteccionMovimientoAR {

    private MandoTelevision mando;

    public DeteccionMovimientoAR(MandoTelevision mando) {
        this.mando = mando;
    }

    public void iniciarDeteccion() throws FrameGrabber.Exception {
        OpenCVFrameConverter.ToMat convertidor = new OpenCVFrameConverter.ToMat();
        VideoCapture captura = new VideoCapture(0);

        if (!captura.isOpened()) {
            System.out.println("No se puede abrir la cámara");
            return;
        }

        Mat fotograma = new Mat();
        Mat fotogramaPrevio = new Mat();
        Mat fotogramaDiferencia = new Mat();
        Mat fotogramaSuavizado = new Mat();
    }

```

```

CanvasFrame lienzo = new CanvasFrame("Detección de Movimiento",
CanvasFrame.getDefaultGamma() / 2.2);
lienzo.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

captura.read(fotogramaPrevio);
opencv_imgproc.cvtColor(fotogramaPrevio, fotogramaPrevio, opencv_imgproc.COLOR_BGR2GRAY);

while (lienzo.isVisible() && captura.read(fotograma)) {
    opencv_imgproc.cvtColor(fotograma, fotograma, opencv_imgproc.COLOR_BGR2GRAY);
    opencv_core.absdiff(fotogramaPrevio, fotograma, fotogramaDiferencia);
    opencv_imgproc.GaussianBlur(fotogramaDiferencia, fotogramaSuavizado, new Size(5, 5), 0);
    opencv_imgproc.threshold(fotogramaSuavizado, fotogramaSuavizado, 25, 255,
opencv_imgproc.THRESH_BINARY);

    MatVector contornos = new MatVector();
    Mat jerarquia = new Mat();
    opencv_imgproc.findContours(fotogramaSuavizado, contornos, jerarquia, opencv_imgproc.RETR_EXTERNAL,
opencv_imgproc.CHAIN_APPROX_SIMPLE);

    boolean brazoIzquierdoLevantado = false;
    boolean brazoDerechoLevantado = false;

    Mat fotogramaConRectangulos = fotograma.clone();

    for (int i = 0; i < contornos.size(); i++) {
        Rect rect = opencv_imgproc.boundingRect(contornos.get(i));
        if (rect.width() < 30 || rect.height() < 30) continue;

        if (rect.y() < fotograma.rows() / 2 && rect.height() > 100) {
            if (rect.x() < fotograma.cols() / 2) {
                brazoIzquierdoLevantado = true;
                opencv_imgproc.rectangle(fotogramaConRectangulos, rect, new Scalar(0, 255, 0, 0));
            } else {
                brazoDerechoLevantado = true;
                opencv_imgproc.rectangle(fotogramaConRectangulos, rect, new Scalar(0, 0, 255, 0));
            }
        }
    }

    if (brazoIzquierdoLevantado) mando.subirVolumen();
    if (brazoDerechoLevantado) mando.bajarVolumen();

    // 1. Obtener el volumen actual , este es un añadido al código del ejercicio que añade la AR
    String textoVolumen = "VOLUMEN: " + mando.obtenerVolumen();

    // 2. Definir la posición y estilo del texto que mostramos en la cámara de realidad aumentada
    Point posicion = new Point(10, 50); // Esquina superior izquierda
    Scalar color = new Scalar(255, 255, 255, 0); // Color Blanco (BGR)
    int fuente = opencv_imgproc.FONT_HERSHEY_DUPLEX; // Tipo de fuente
    double escala = 1.5; // Tamaño de la fuente
    int grosor = 2; // Grosor de la línea

    // 3. Dibujar el texto del volumen en el fotograma de AR
    opencv_imgproc.putText(fotogramaConRectangulos, textoVolumen, posicion, fuente, escala, color, grosor,
opencv_imgproc.LINE_AA, false);

    Frame fotogramaMostrar = convertidor.convert(fotogramaConRectangulos);
    lienzo.showImage(fotogramaMostrar);

    if (contornos.size() > 0) fotogramaPrevio = fotograma.clone();
    if (mando.getSalir()) { // Si la voz pidió salir entonces ejecutamos el break para la cámara también
        break; // Sale del bucle de la cámara
    }
}

captura.release();

```

```

        lienzo.dispose();
    }
}

```

DeteccionMovimientoAR.java es una clase que gestiona el proceso en el que la camara se enciende, el programa , captura el fotograma y lo convierte a escala de grises , entonces se pone a calcular la diferencia absoluta con el fotograma previo para aislar el movimiento , aplicar un suavizado Gaussiano , y luego un umbral (binarización). Finalmente, se buscan los contornos para identificar y delimitar los rectángulos de los brazos en la mitad superior de la pantalla. En caso de levantar la mano izquierda o derecha, sube o jaba el volumen, ademas se encarga de mostrar en pantalla el volumen al que esta con realidad aumentada por lo que como explico en el codigo al principio importamos todas las librerias necesarias tanto para mostrar el texto del volumen, como para gestionar el modelo de reconocimiento de imagenes , todo lo necesario para que funcione perfectamente . La mayoría del codigo ya lo teniamos por que se nos habia entregado pero hay unas pequeñas modificaciones como lo de la realidad aumentada por ejemplo que le dan un mejor funcionamiento.

MandoVoz.java

```

// Paquete para agrupar todas las clases que vamos a utilizar en nuestro proyecto
package com.example.mandoar;

// Importamos vosk y las clases que necesitamos para la gestion del reconocimiento de voz
import org.vosk.Model;
import org.vosk.Recognizer;
import org.vosk.LibVosk;
import org.vosk.LogLevel;

// Importamos la libreria de javax.sound.sampled para capturar el audio
import javax.sound.sampled.*;
// importamos la libreria para la gestion de las excepciones
import java.io.IOException;

// clase principal MandoVoz para reconocer la voz, escribirla en pantalla y ejecutar acciones en el mando de televisión
public class MandoVoz {

    // Atributo MandoTelevision que es el que va a gestionar
    private final MandoTelevision mando;

    // Constructor que recibe el mando de la televisión que controlaremos con la voz
    public MandoVoz(MandoTelevision mando) {
        this.mando = mando;
    }

    // Metodo iniciarReconocimiento que es el que activa el reconocimiento de voz, activa microfono, // incia vosk, y
    // procesa los comandos de voz

    public void iniciarReconocimiento() throws IOException, LineUnavailableException {
        // Activamos el nivel de log de Vosk para obtener información
        LibVosk.setLogLevel(LogLevel.INFO);

        // Cargamos el modelo de reconocimiento de voz en español que hemos instalado en los resources
        Model model = new Model("src/main/resources/model/vosk-model-small-es-0.42");
        // Creamos el reconocedor con el modelo cargado y una frecuencia determinada
        Recognizer recognizer = new Recognizer(model, 16000);

        // Definimos el formato del audio que recibiremos en el micro
        AudioFormat formato = new AudioFormat(16000, 16, 1, true, false);
        // Creamos el canal de audio usando la configuracion que hemos dado previamente en el formato
        DataLine.Info info = new DataLine.Info(TargetDataLine.class, formato);
        // Creamos el canal de audio usando la configuracion que hemos indicado previamente
        TargetDataLine linea = (TargetDataLine) AudioSystem.getLine(info);
    }
}

```



```

// Abrimos la linea para que se capture el audio
linea.open(formato);
// Iniciamos la captura de audio
linea.start();

// Buffer para almacenar los trozos de audio que leemos
byte[] buffer = new byte[4096];
// Aqui caumulamos lo que se ha dicho
StringBuilder textoAcumulado = new StringBuilder();

// Imprimimos en pantalla las indicaciones de lo que podemos hacer y como parar
System.out.println("Di 'subir volumen', 'bajar volumen' o 'parar' para detener...");

try {
    while (true) {
        // Leemos audio del micro y lo almacenamos en el buffer
        int bytesLeidos = linea.read(buffer, 0, buffer.length);
        if (bytesLeidos <= 0) continue;
        // Si Vosk detecta un resultado completo procesamos el texto reconocido
        if (recognizer.acceptWaveForm(buffer, bytesLeidos)) {
            String resultado = recognizer.getResult();
            String texto = extraerTexto(resultado);
            System.out.println("Comando por voz: " + texto);
            // Acumulamos todas las palabras reconocidas
            textoAcumulado.append(texto).append(" ");

            // Comandos que se pueden reconcer ...
            // Si se reconoce que hay que subir el volumen ejecutamos el metodo subirvolumen de la clase mando television
            if (textoAcumulado.toString().toLowerCase().contains("subir volumen")) {
                mando.subirVolumen();
                // Indicamos en pantalla el volumen que ha quedado como resultado y y que hemos subido el volumen
                System.out.println("Volumen subido: " + mando.obtenerVolumen());
            }
            // Si se reconoce que hay que bajar el volumen ejecutamos el metodo bajarvolumen de la clase mando television
            } else if (textoAcumulado.toString().toLowerCase().contains("bajar volumen")) {
                mando.bajarVolumen();
                // Indicamos en pantalla que hemos bajado el volumen y el volumen resultante
                System.out.println("Volumen bajado: " + mando.obtenerVolumen());
            }
            } else if (textoAcumulado.toString().toLowerCase().contains("para")) {
                // Indicamos que acabamos de recibir la indicacion de parar
                System.out.println("\nComando 'para' detectado. ¡Deteniendo reconocimiento de voz!");
                mando.setSalir(true);
                break; // Sale del bucle de voz
            }
        }
    }
} finally {
    // Cierre seguro del microfono
    if (linea.isOpen()) {
        linea.stop();
        linea.close();
        System.out.println("Micrófono cerrado correctamente.");
    }
    // Cerramos el modelo de vosk
    model.close();
    // Indicamos que el reconocimiento de voz ha finalizado
    System.out.println("Reconocimiento de voz finalizado.");
}
}

// Extrae el texto del JSON que devuelve Vosk
// Ejemplo de entrada del JSON : {"text" : "hola"} entonces devuelve solo "hola" es mas limpio
private String extraerTexto(String json) {
    int start = json.indexOf("\"text\" : \"");
    if (start == -1) return "";
    start += 10;
    int end = json.indexOf("\"", start);

```

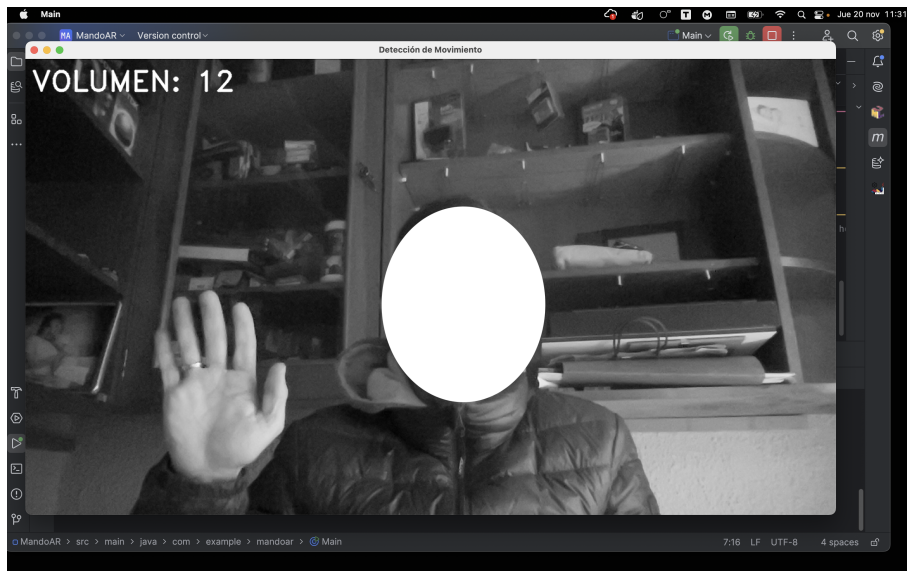
```

    return end > start ? json.substring(start, end) : "";
}

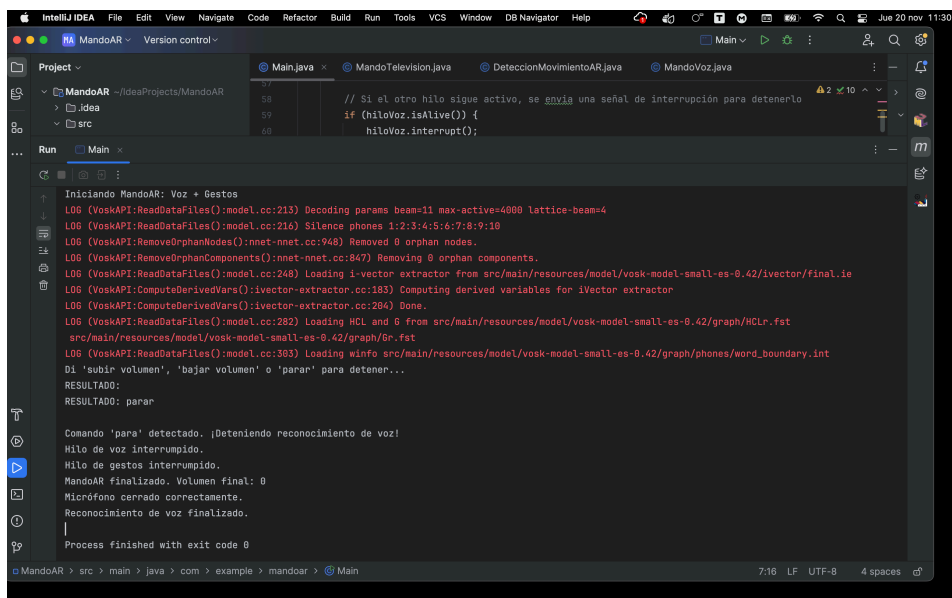
```

Y por ultimo MandoVoz.java que es la clase encargada de detectar lo que decimos a traves del microfono, por lo que tenemos que poner en marcha el microfono con unos parametros determinados , a partir de ahi si decimos subir volumen y lo reconoce el modelo de reonomiento de voz sube 1 el volumen, y si reconoce bajar volumen baja en un 1 punto el volumen.

Añado captura de pantalla del funcionamiento del programa :



Y aqui mostramos una captura de pantalla de como cierra la ejecución cuando decimos parar por voz :

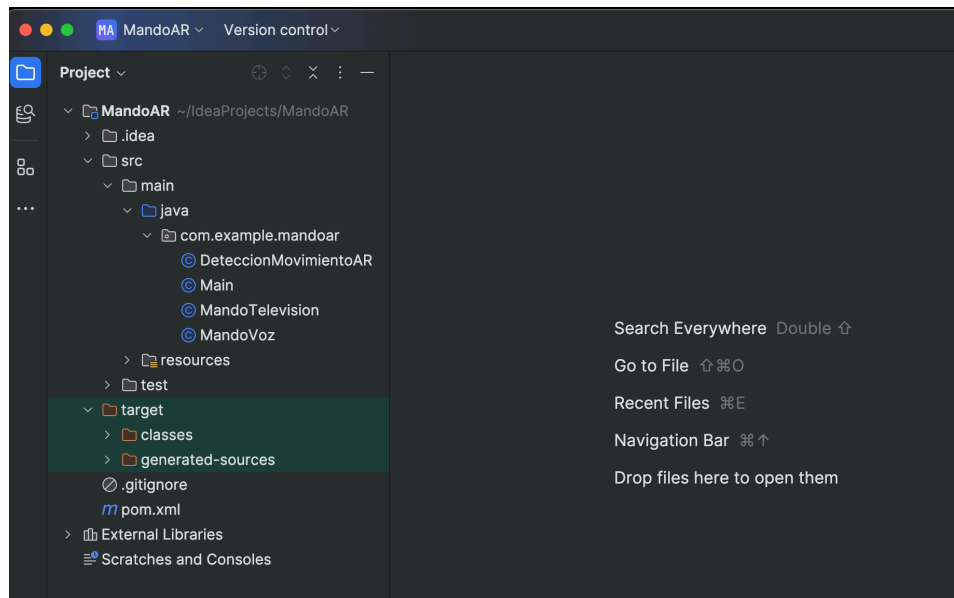


Importante !! Adjunto enlace a drive para ver toda la carpeta del proyecto :

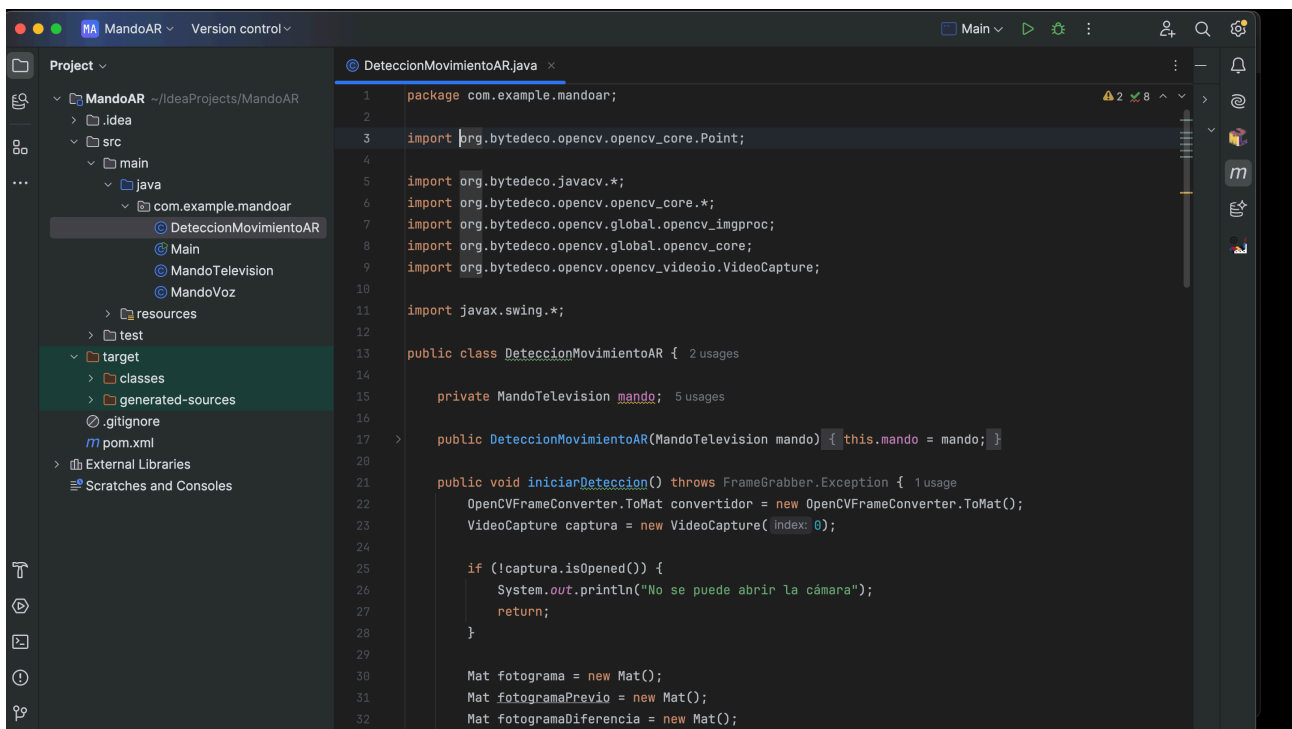
https://drive.google.com/drive/folders/1ZxKWS73Wj2YKpfGc4VQx930hH0pnnvGSt?usp=drive_link

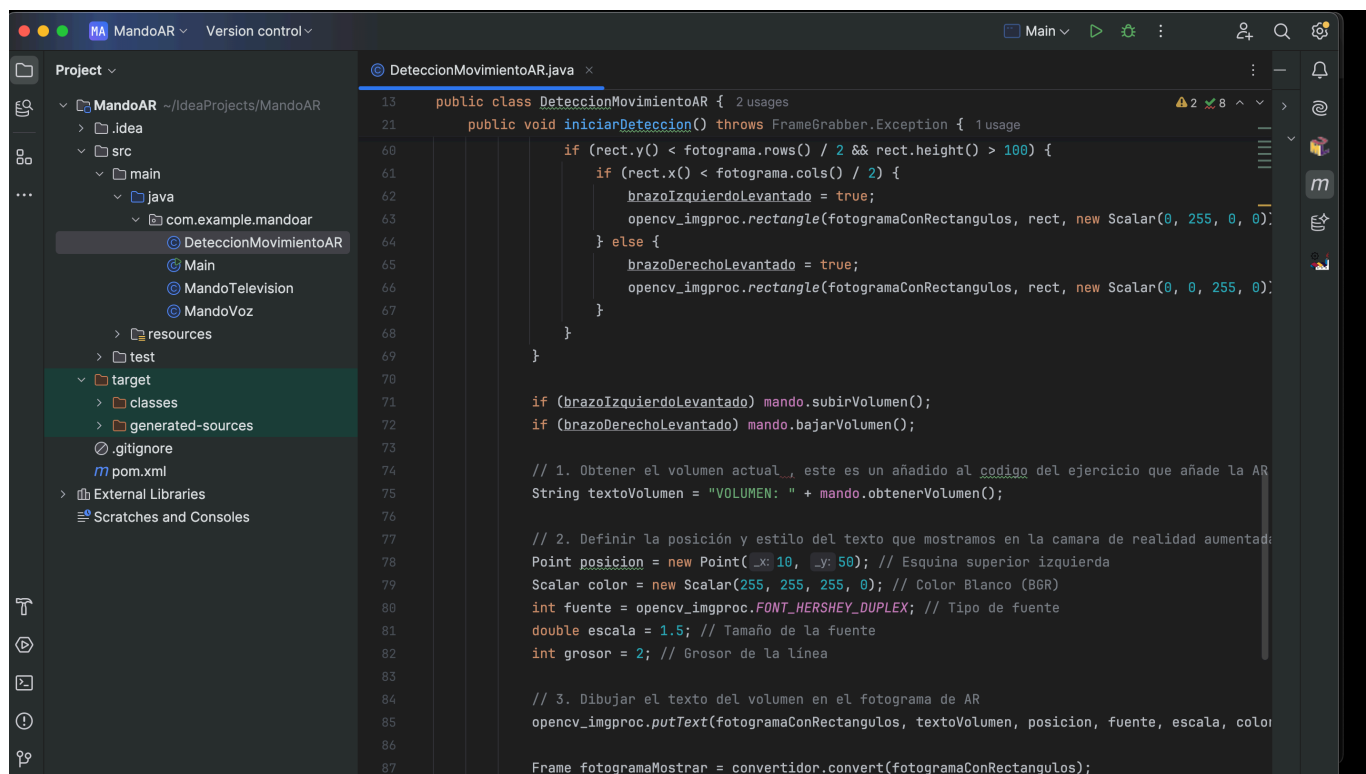
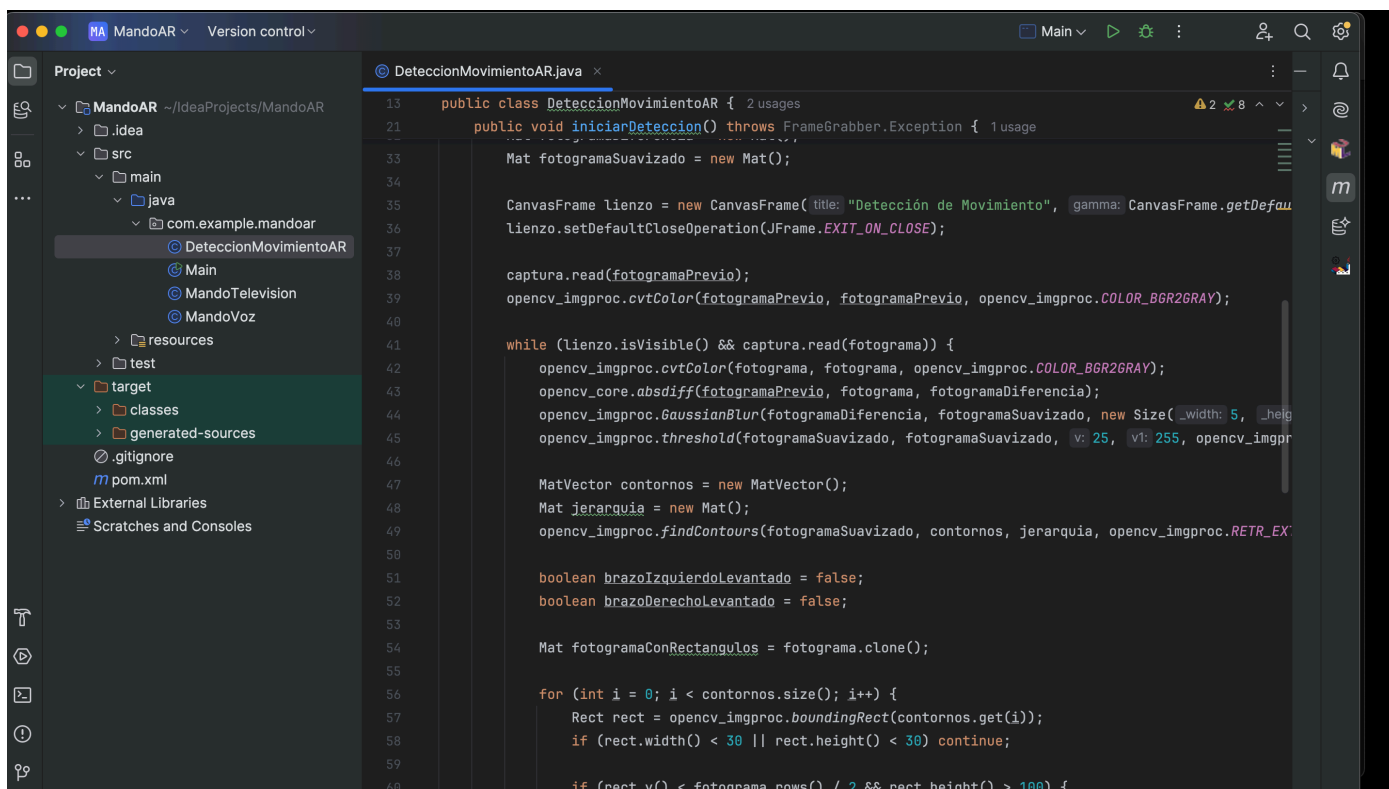
Aquí quiero añadir unas últimas capturas de pantalla con muestras del código en IntelliJ para visualizar un poco como es todo el código, aunque lo he añadido arriba en el pdf aquí se puede comprobar muy bien la estructura del proyecto y todo el código en el IDE.

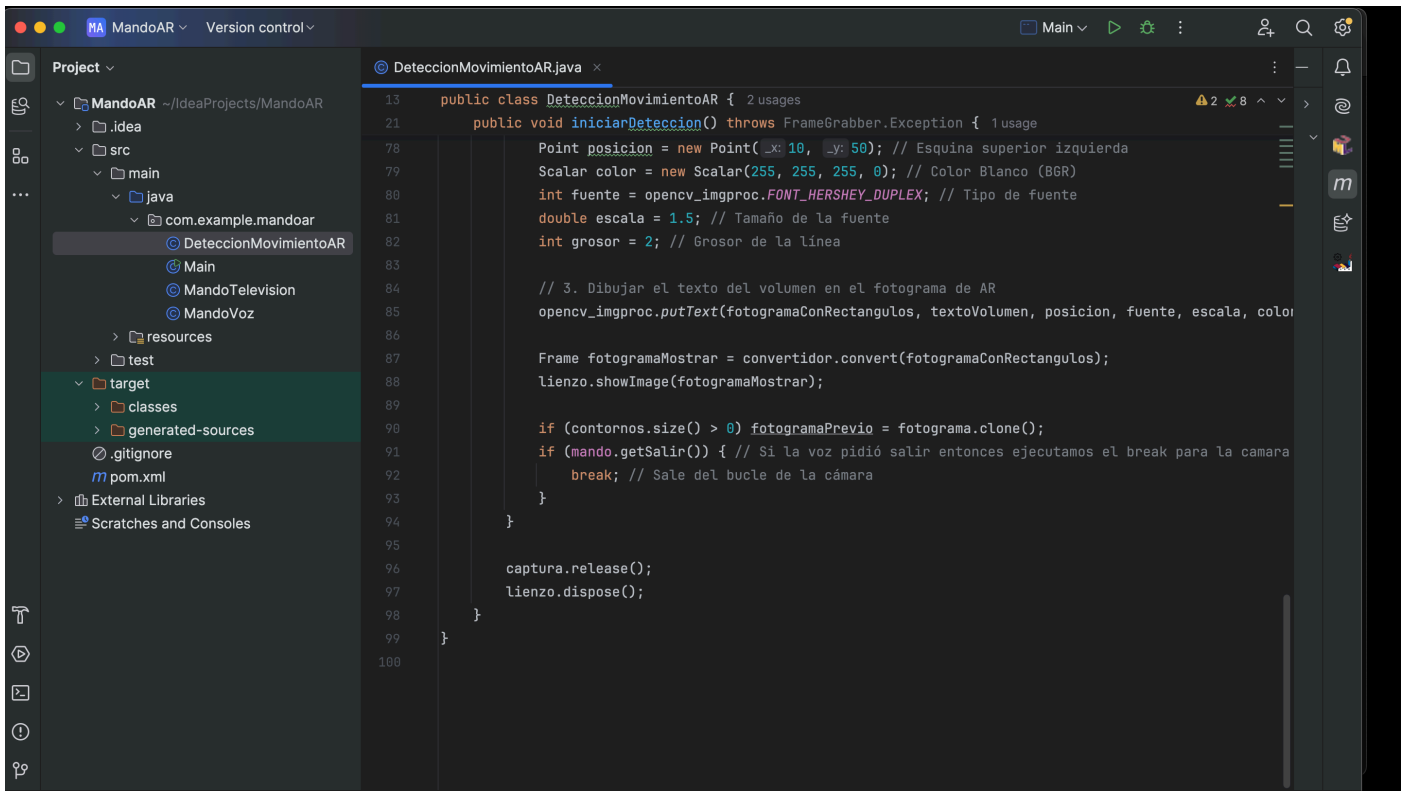
Proyecto :



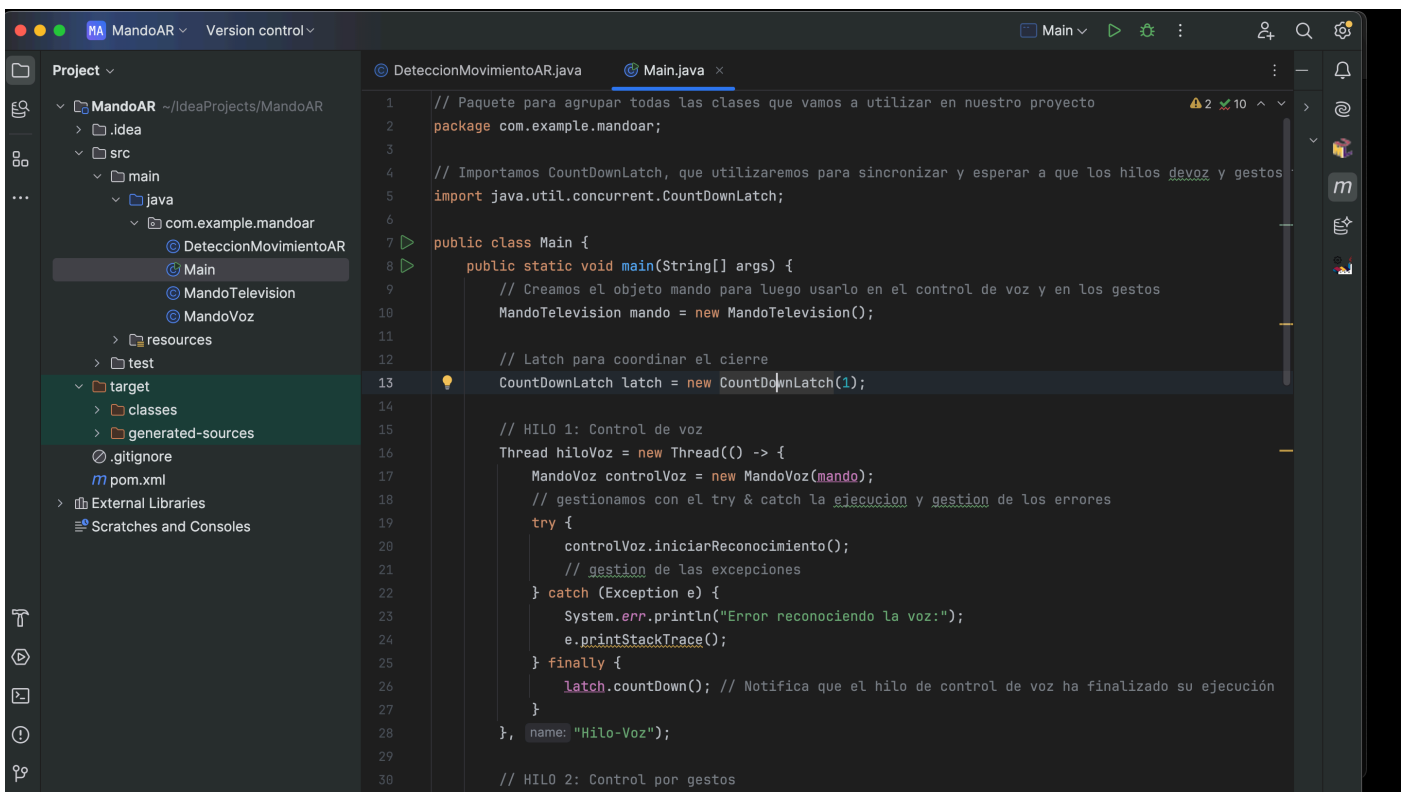
DeteccionMovimientoAR.java :

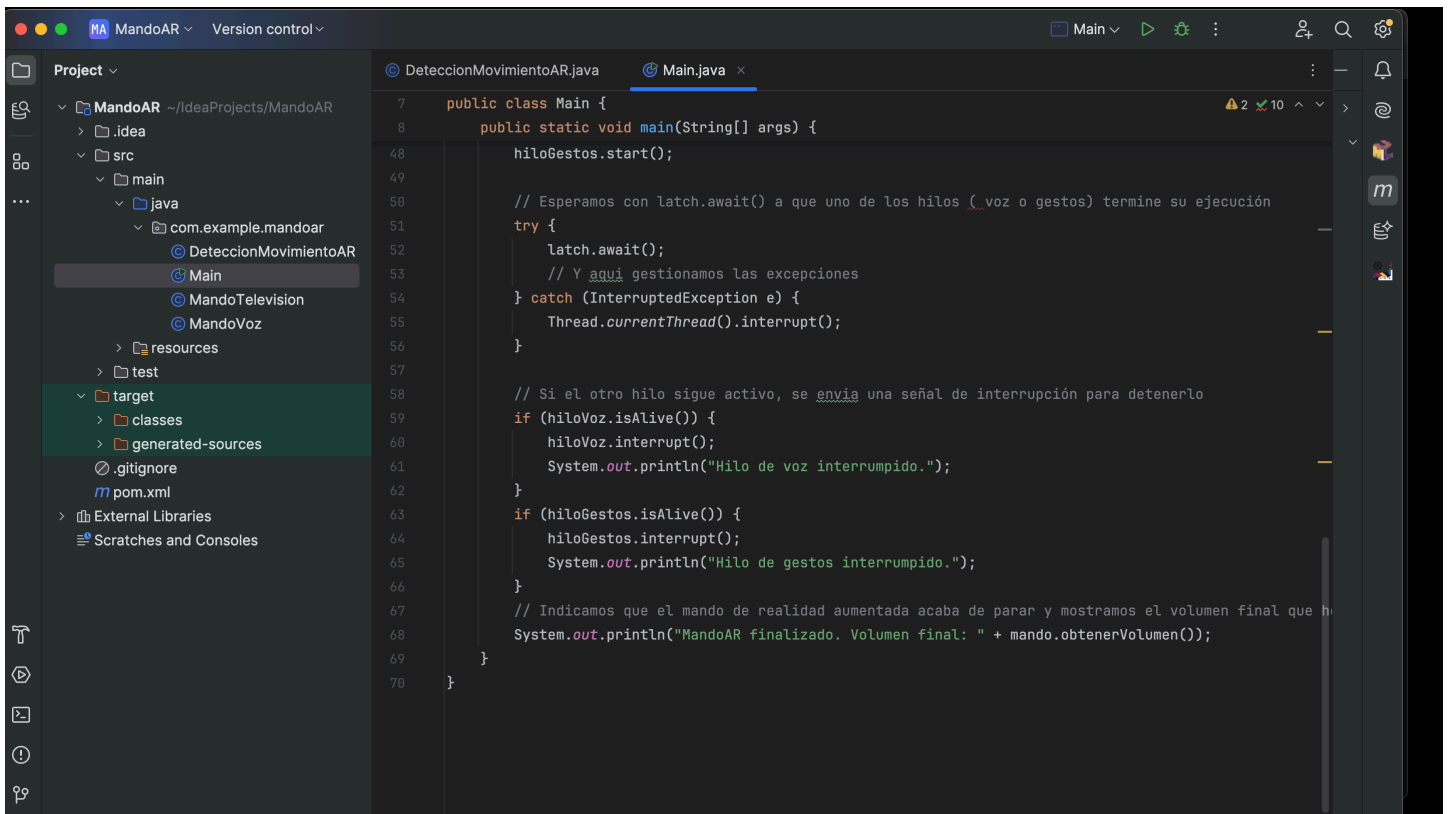
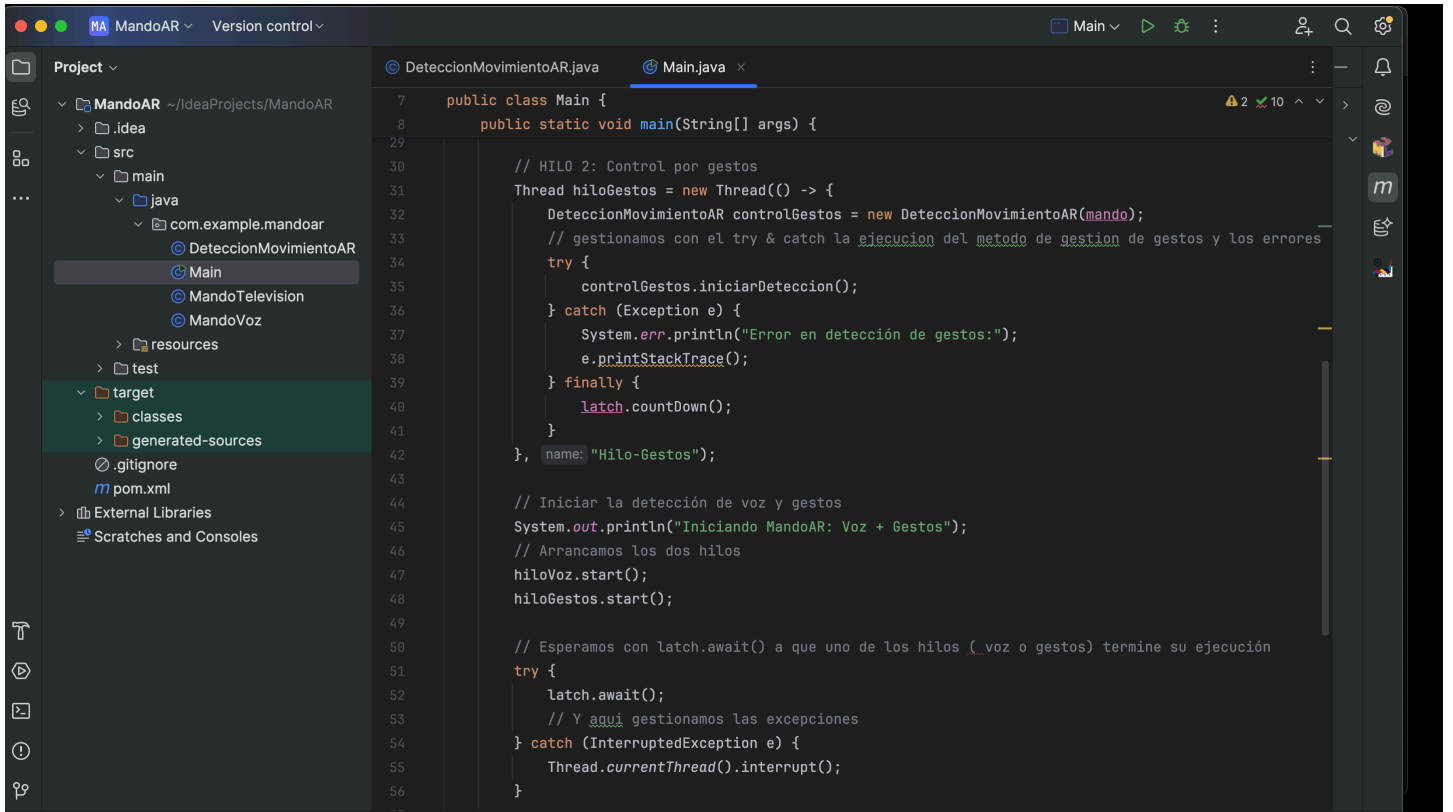




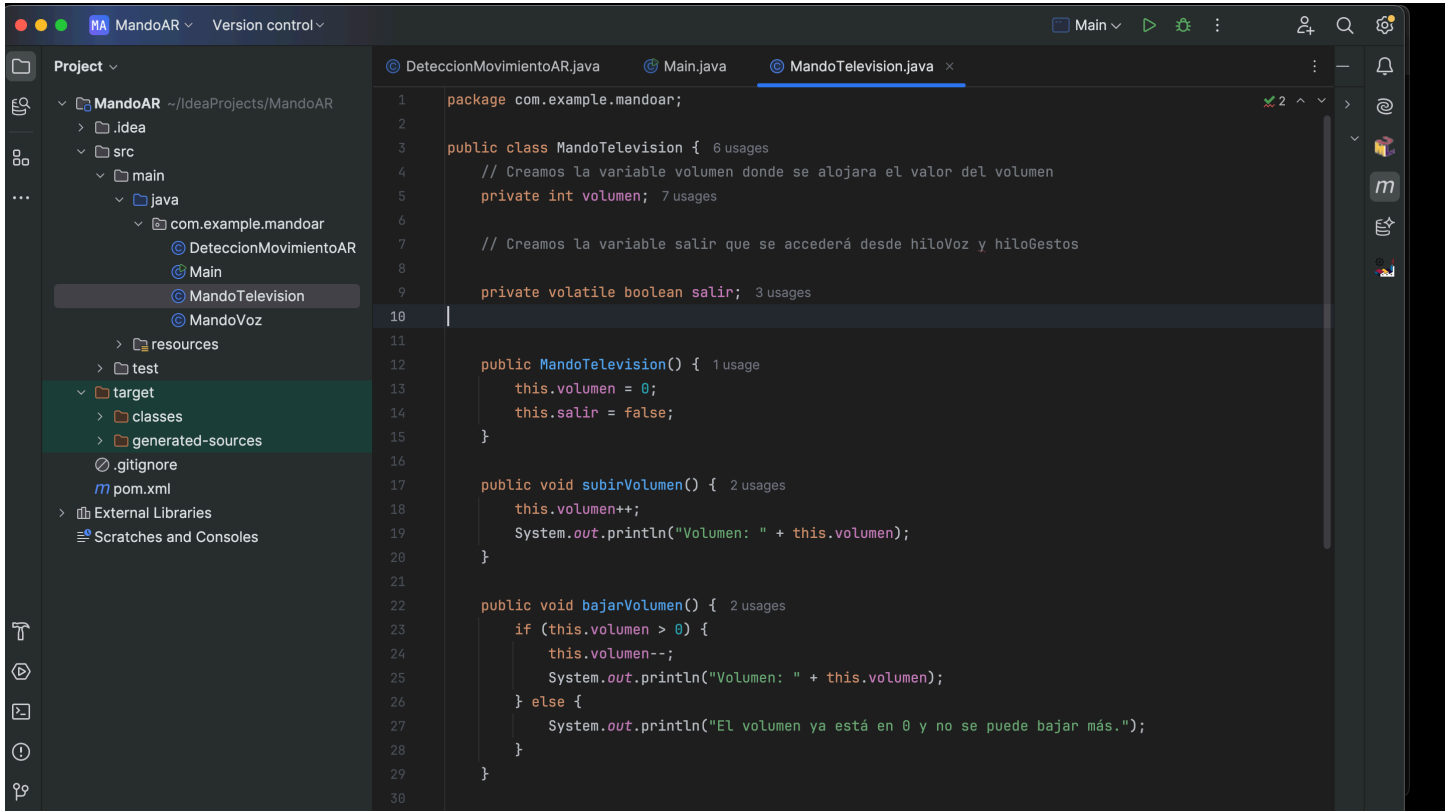


Main.java :

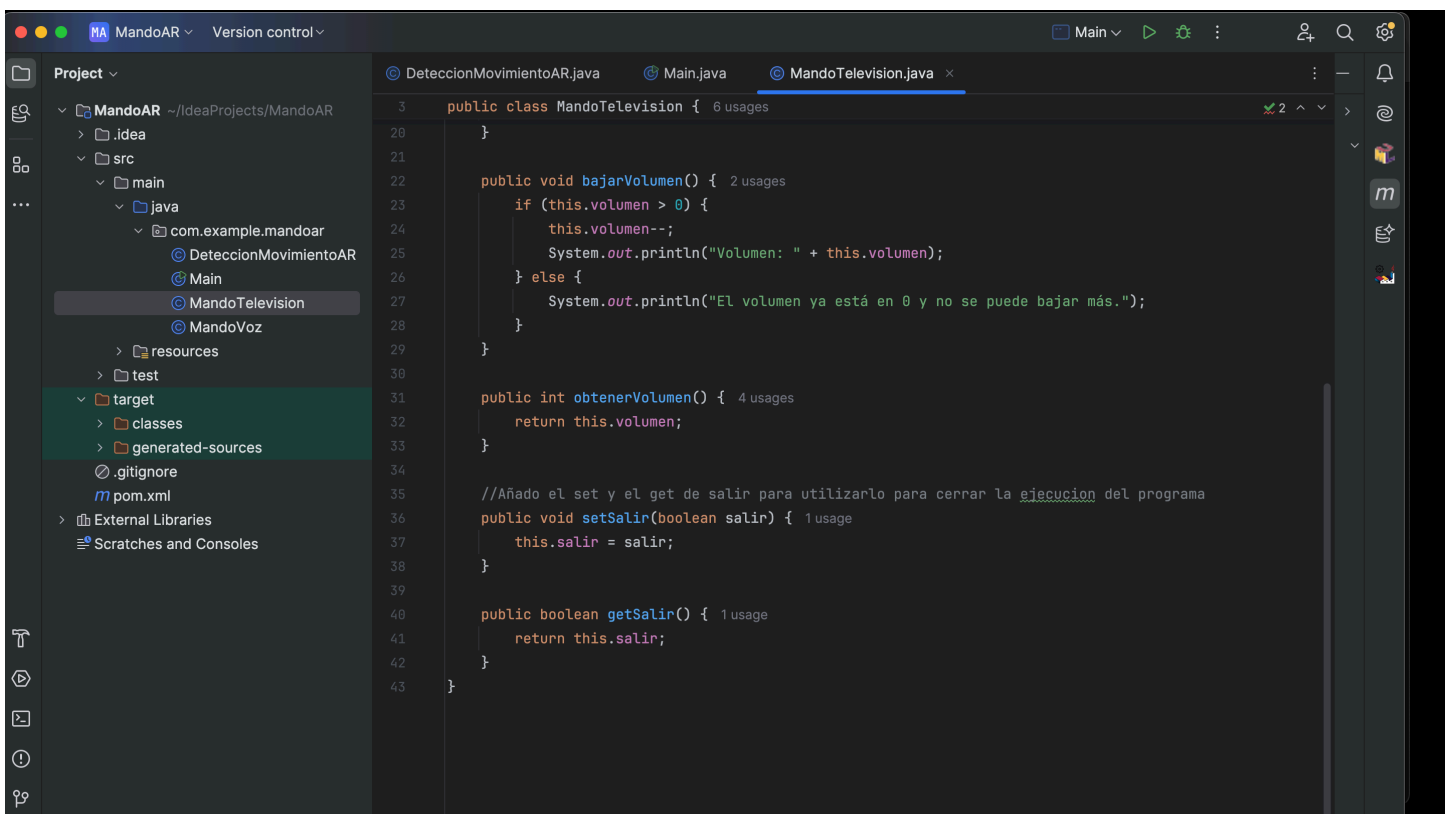




MandoTelevision.java

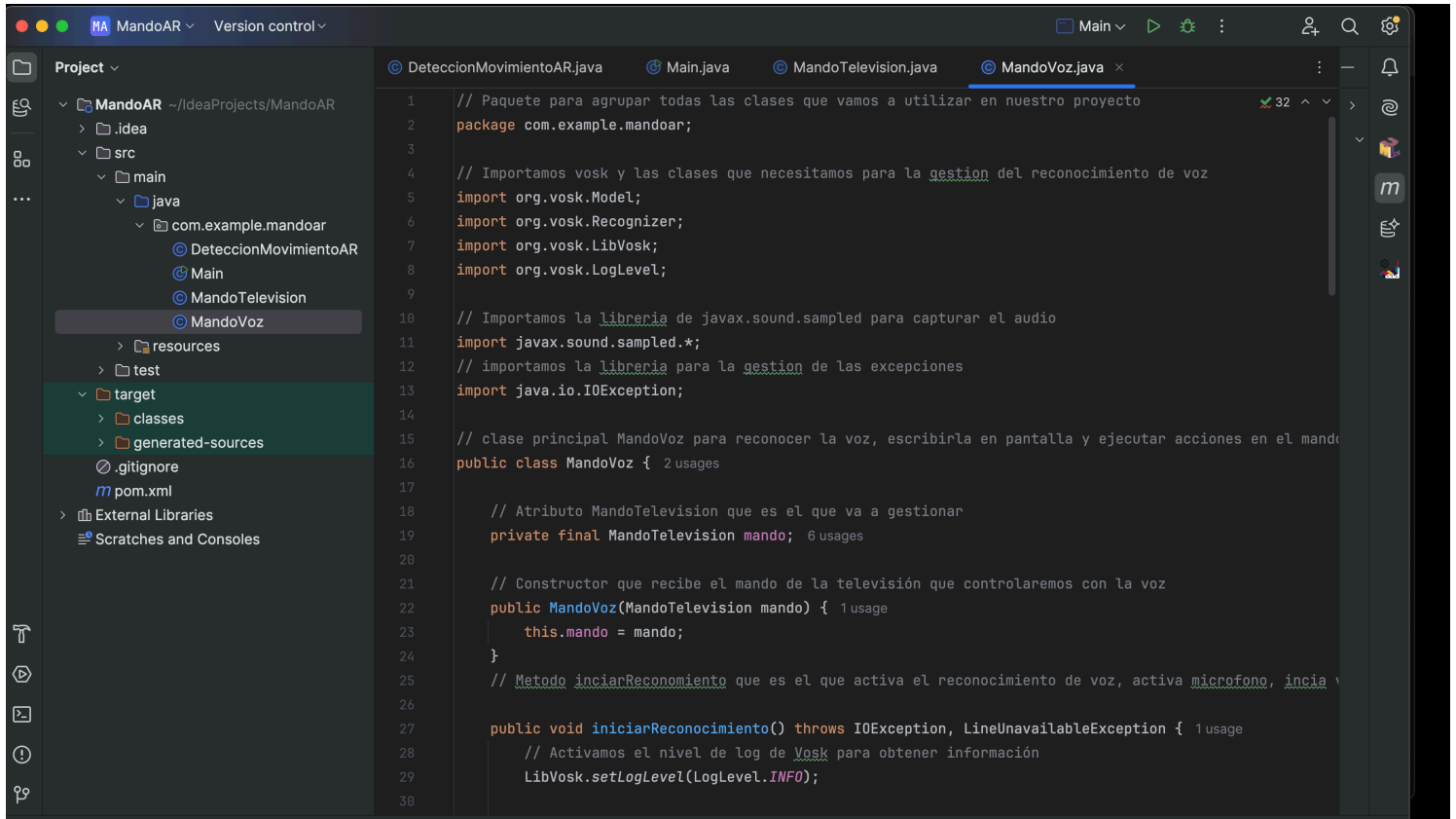


```
1 package com.example.mandoar;
2
3 public class MandoTelevision { 6 usages
4     // Creamos la variable volumen donde se alojara el valor del volumen
5     private int volumen; 7 usages
6
7     // Creamos la variable salir que se accederá desde hiloVoz y hiloGestos
8
9     private volatile boolean salir; 3 usages
10
11
12     public MandoTelevision() { 1 usage
13         this.volumen = 0;
14         this.salir = false;
15     }
16
17     public void subirVolumen() { 2 usages
18         this.volumen++;
19         System.out.println("Volumen: " + this.volumen);
20     }
21
22     public void bajarVolumen() { 2 usages
23         if (this.volumen > 0) {
24             this.volumen--;
25             System.out.println("Volumen: " + this.volumen);
26         } else {
27             System.out.println("El volumen ya está en 0 y no se puede bajar más.");
28         }
29     }
30 }
```

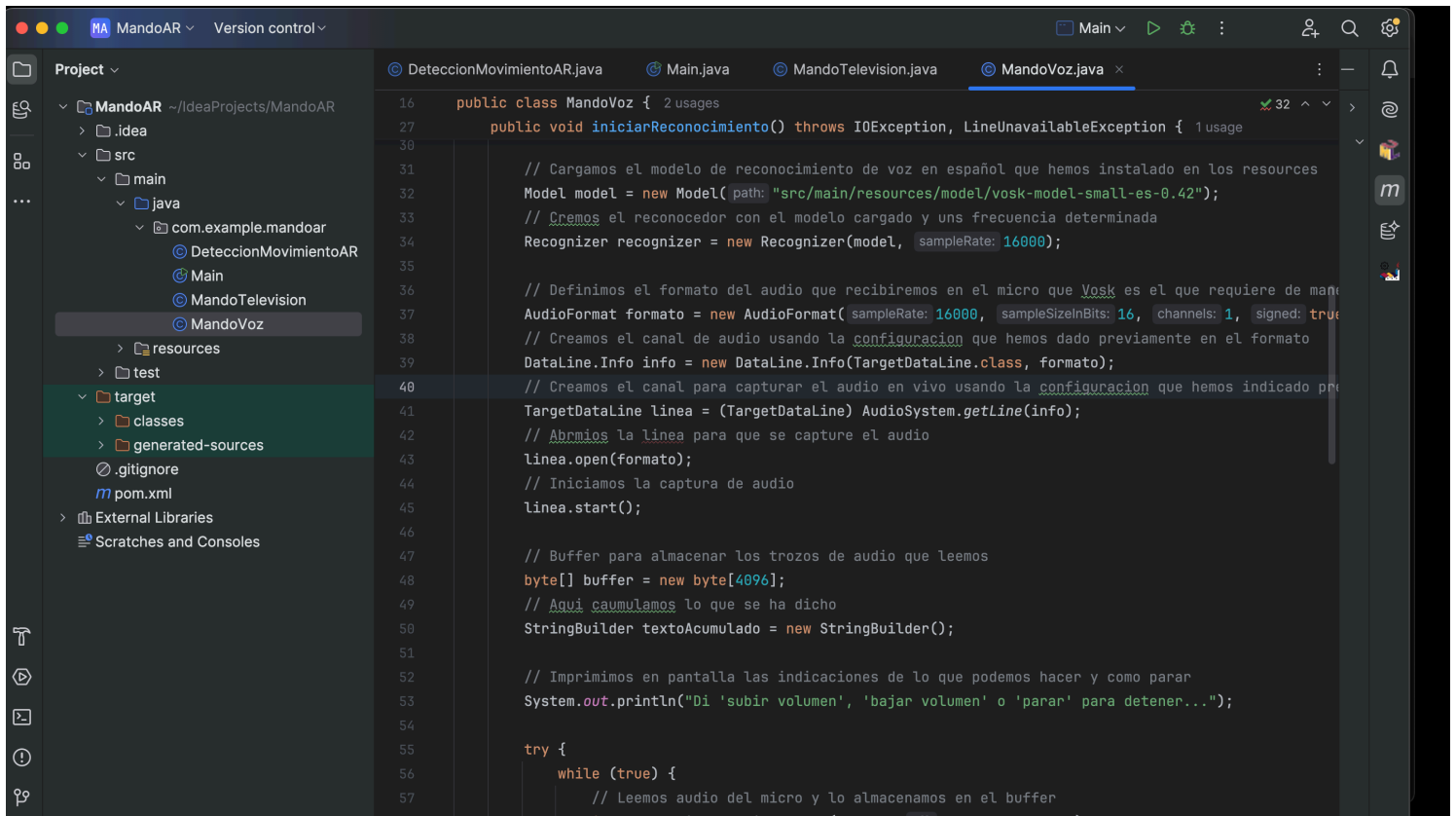


```
3 public class MandoTelevision { 6 usages
20 }
21
22     public void bajarVolumen() { 2 usages
23         if (this.volumen > 0) {
24             this.volumen--;
25             System.out.println("Volumen: " + this.volumen);
26         } else {
27             System.out.println("El volumen ya está en 0 y no se puede bajar más.");
28         }
29     }
30
31     public int obtenerVolumen() { 4 usages
32         return this.volumen;
33     }
34
35     //Añado el set y el get de salir para utilizarlo para cerrar la ejecución del programa
36     public void setSalir(boolean salir) { 1 usage
37         this.salir = salir;
38     }
39
40     public boolean getSalir() { 1 usage
41         return this.salir;
42     }
43 }
```

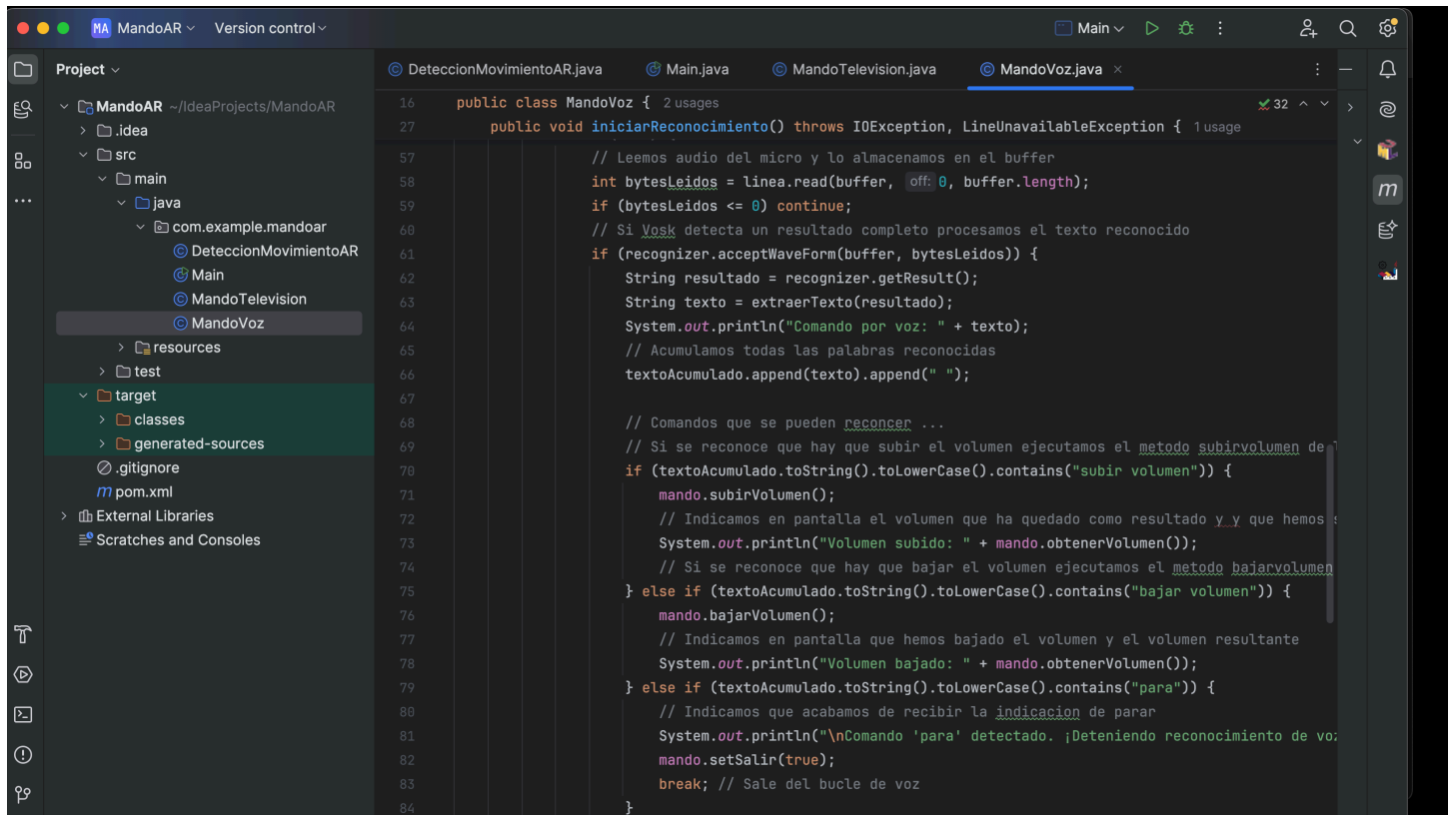

MandoVoz.java



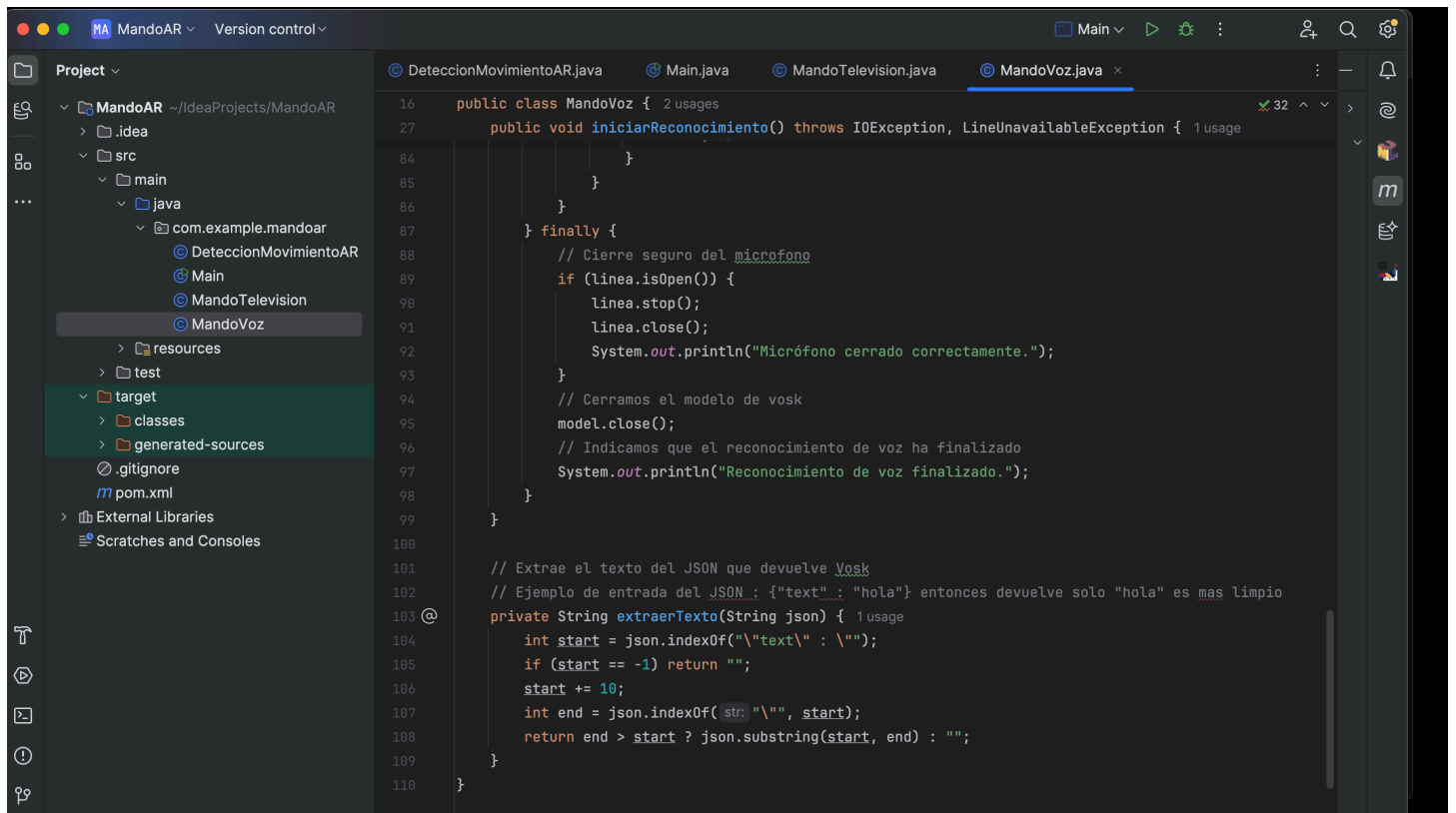
```
1 // Paquete para agrupar todas las clases que vamos a utilizar en nuestro proyecto
2 package com.example.mandoar;
3
4 // Importamos vosk y las clases que necesitamos para la gestion del reconocimiento de voz
5 import org.vosk.Model;
6 import org.vosk.Recognizer;
7 import org.vosk.LibVosk;
8 import org.vosk.LogLevel;
9
10 // Importamos la libreria de javax.sound.sampled para capturar el audio
11 import javax.sound.sampled.*;
12 // importamos la libreria para la gestion de las excepciones
13 import java.io.IOException;
14
15 // clase principal MandoVoz para reconocer la voz, escribirla en pantalla y ejecutar acciones en el mando
16 public class MandoVoz { 2 usages
17
18     // Atributo MandoTelevision que es el que va a gestionar
19     private final MandoTelevision mando; 6 usages
20
21     // Constructor que recibe el mando de la television que controlaremos con la voz
22     public MandoVoz(MandoTelevision mando) { 1 usage
23         this.mando = mando;
24     }
25
26     // Metodo iniciarReconocimiento que es el que activa el reconocimiento de voz, activa microfono, inicia
27
28     public void iniciarReconocimiento() throws IOException, LineUnavailableException { 1 usage
29         // Activamos el nivel de log de Vosk para obtener información
30         LibVosk.setLogLevel(LogLevel.INFO);
```



```
16 public class MandoVoz { 2 usages
27     public void iniciarReconocimiento() throws IOException, LineUnavailableException { 1 usage
30
31         // Cargamos el modelo de reconocimiento de voz en español que hemos instalado en los recursos
32         Model model = new Model( path: "src/main/resources/model/vosk-model-small-es-0.42");
33         // Creamos el reconocedor con el modelo cargado y una frecuencia determinada
34         Recognizer recognizer = new Recognizer(model, sampleRate: 16000);
35
36         // Definimos el formato del audio que recibiremos en el micro que Vosk es el que requiere de manejar
37         AudioFormat formato = new AudioFormat( sampleRate: 16000, sampleSizeInBits: 16, channels: 1, signed: true);
38         // Creamos el canal de audio usando la configuracion que hemos dado previamente en el formato
39         DataLine.Info info = new DataLine.Info(TargetDataLine.class, formato);
40         // Creamos el canal para capturar el audio en vivo usando la configuracion que hemos indicado previamente
41         TargetDataLine linea = (TargetDataLine) AudioSystem.getLine(info);
42         // Abrimos la linea para que se capture el audio
43         linea.open(formato);
44         // Iniciamos la captura de audio
45         linea.start();
46
47         // Buffer para almacenar los trozos de audio que leemos
48         byte[] buffer = new byte[4096];
49         // Aqui acumulamos lo que se ha dicho
50         StringBuilder textoAcumulado = new StringBuilder();
51
52         // Imprimimos en pantalla las indicaciones de lo que podemos hacer y como parar
53         System.out.println("Di 'subir volumen', 'bajar volumen' o 'parar' para detener...");
54
55         try {
56             while (true) {
57                 // Leemos audio del micro y lo almacenamos en el buffer
```



```
16 public class MandoVoz { 2 usages
27 public void iniciarReconocimiento() throws IOException, LineUnavailableException { 1 usage
57 // Leemos audio del micro y lo almacenamos en el buffer
58 int bytesLeídos = linea.read(buffer, off: 0, buffer.length);
59 if (bytesLeídos <= 0) continue;
60 // Si Vosk detecta un resultado completo procesamos el texto reconocido
61 if (recognizer.acceptWaveForm(buffer, bytesLeídos)) {
62     String resultado = recognizer.getResult();
63     String texto = extraerTexto(resultado);
64     System.out.println("Comando por voz: " + texto);
65     // Acumulamos todas las palabras reconocidas
66     textoAcumulado.append(texto).append(" ");
67
68     // Comandos que se pueden reconocer ...
69     // Si se reconoce que hay que subir el volumen ejecutamos el metodo subirvolumen de
70     if (textoAcumulado.toString().toLowerCase().contains("subir volumen")) {
71         mando.subirVolumen();
72         // Indicamos en pantalla el volumen que ha quedado como resultado y y que hemos s
73         System.out.println("Volumen subido: " + mando.obtenerVolumen());
74         // Si se reconoce que hay que bajar el volumen ejecutamos el metodo bajarvolumen
75     } else if (textoAcumulado.toString().toLowerCase().contains("bajar volumen")) {
76         mando.bajarVolumen();
77         // Indicamos en pantalla que hemos bajado el volumen y el volumen resultante
78         System.out.println("Volumen bajado: " + mando.obtenerVolumen());
79     } else if (textoAcumulado.toString().toLowerCase().contains("para")) {
80         // Indicamos que acabamos de recibir la indicacion de parar
81         System.out.println("\nComando 'para' detectado. ¡Deteniendo reconocimiento de voz!
82         mando.setSalir(true);
83         break; // Sale del bucle de voz
84     }
```



```
84     }
85 }
86 }
87 } finally {
88     // Cierre seguro del microfono
89     if (linea.isOpen()) {
90         linea.stop();
91         linea.close();
92         System.out.println("Micrófono cerrado correctamente.");
93     }
94     // Cerramos el modelo de vosk
95     model.close();
96     // Indicamos que el reconocimiento de voz ha finalizado
97     System.out.println("Reconocimiento de voz finalizado.");
98 }
99 }
100
101 // Extrae el texto del JSON que devuelve Vosk
102 // Ejemplo de entrada del JSON : {"text": "hola"} entonces devuelve solo "hola" es mas limpio
103 @ private String extraerTexto(String json) { 1 usage
104     int start = json.indexOf("\"text\" : \"");
105     if (start == -1) return "";
106     start += 10;
107     int end = json.indexOf("\"", start);
108     return end > start ? json.substring(start, end) : "";
109 }
110 }
```